Hafta 3: Modern C++ ve STL Uygulamaları

Yazılım Geliştirme II – Çalışma Kağıdı

Ad Soyad: _____	Numara: _____	Tarih: _____

Yönergeler

- Bu çalışma kağıdı **6 bölümden** oluşmaktadır.
- Çoktan seçmeli sorularda doğru seçeneği **daire içine alın**.
- Boşluk doldurma sorularında cevabınızı **çizgi üzerine** yazın.
- Açık uçlu sorularda cevabınızı **verilen alana** yazın.
- Süre: **40 dakika**

Bölüm A: Çoktan Seçmeli Sorular

1. `std::ranges::sort(v)` ile `std::sort(v.begin(), v.end())` arasındaki temel fark nedir? **2 puan**
   A) `ranges::sort` daha yavaş çalışır
   B) `ranges::sort` doğrudan aralık (range) kabul eder, iteratör çifti gerektirmez
   C) Aynıdırlar, sadece isim alanı farklıdır
   D) `ranges::sort` sadece `vector` ile çalışır

2. `std::string_view` nesnesi ne zaman tehlikelidir? **2 puan**
   A) Kısa stringler ile kullanıldığında
   B) Const referans yerine kullanıldığında
   C) Kaynak string yok edildikten sonra kullanıldığında (dangling)
   D) `std::string` ile birlikte kullanıldığında

3. `std::format("{:>10}", "test")` ifadesinin çıktısı nedir? **2 puan**
   A) `test` (sola hizalı)
   B) `      test` (sağa hizalı, 10 karakter)
   C) `   test   ` (ortalı)
   D) Derleme hatası

4. `std::optional<int> x;` tanımında `x.value()` çağrılırsa ne olur? **2 puan**
   A) 0 döndürür
   B) `std::bad_optional_access` istisnası fırlatır
   C) Tanımsız davranış oluşur
   D) Program sessizce sonlanır

5. `std::variant<int, double, std::string>` içindeki aktif türü öğrenmek için ne kullanılır? **2 puan**
   A) `v.type()`
   B) `v.index()` ve `std::holds_alternative<T>(v)`
   C) `typeid(v)`
   D) `v.which()`

6. Aşağıdaki views pipeline'ın sonucu nedir? **2 puan**

```
std::vector<int> v = {1, 2, 3, 4, 5, 6};
auto r = v | std::views::filter([](int x){ return x % 2 == 0; })
           | std::views::transform([](int x){ return x * x; });
// r uzerinde dolasim
```

A) `1 4 9 16 25 36`
B) `4 16 36`
C) `2 4 6`
D) `1 9 25`

7. `noexcept` belirteci bir fonksiyona eklendiğinde ne olur? **2 puan**

A) Fonksiyon istisna fırlatamaz, fırlatırsa `std::terminate` çağrılır
B) Fonksiyon otomatik olarak try-catch ile sarılır
C) Derleme zamanında istisna kontrolü yapılır
D) Fonksiyonun performansı değişmez

8. C++20 projeksiyonlarının amacı nedir? **2 puan**

A) Verileri veritabanına yansıtmak
B) Sıralama/arama öncesinde elemanlara bir dönüşüm uygulamak
C) Veri tiplerini otomatik dönüştürmek
D) Paralel hesaplama yapmak

Bölüm B: Doğru / Yanlış

Her ifadenin doğru (D) veya yanlış (Y) olduğunu belirtin.

1. `std::views::filter` tembel (lazy) değerlendirme yapar, veriyi anında kopyalamaz.
2. `std::string_view` bir string kopyası oluşturur.
3. `std::format` tür güvenli (type-safe) biçimlendirme sağlar.
4. `std::optional` en fazla bir değer tutabilir veya boş olabilir.
5. `std::variant` aynı anda birden fazla türde değer tutabilir.
6. `std::visit` ile variant içindeki değere tür güvenli erişim sağlanır.

Bölüm C: Eşleştirme

Sol sütundaki kavramları sağ sütundaki açıklamalarla eşleştirin.

#	Kavram	Cevap	#	Açıklama
1	`views::filter`		a	Belirli sayıda eleman al
2	`views::transform`		b	Koşulu sağlayan elemanları geçir
3	`views::take`		c	Her elemana dönüşüm uygula
4	`views::drop`		d	İlk n elemanı atla
5	`views::reverse`		e	Ters sırada görüntüle

Bölüm D: Boşluk Doldurma

Aşağıdaki cümlelerde boş bırakılan yerleri uygun ifadelerle doldurun.

1. C++20 `std::ranges` kütüphanesi, iteratör çiftleri yerine doğrudan _____ ile çalışmayı sağlar.
2. `std::string_view` bir string'e _____ (kopyalamayan) bir görünüm sağlar.
3. `std::format` fonksiyonu `printf`'in modern, _____ alternatifidir.
4. `std::optional` "değer yok" durumunu `std::nullopt` veya _____ ile ifade eder.
5. `std::variant` C birleşimlerinin (union) _____ alternatifidir.
6. Pipe operatörü (`|`) ile views'ler _____ oluşturmak için zincirlenir.
7. `std::visit` fonksiyonu variant üzerinde _____ deseni uygular.

Bölüm E: Kod Çıktısı

Aşağıdaki C++ kodlarının çıktılarını yazın.

1. **3 puan**

```
std::vector<int> v = {5, 3, 8, 1, 9};
std::ranges::sort(v);
for (int x : v) std::cout << x << " ";
```

Çıktı:

2. **3 puan**

```
std::optional<int> a = 42;
std::optional<int> b;
std::cout << a.value_or(0) << " " << b.value_or(-1);
```

Çıktı:

3. **4 puan**

```
std::vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8};
auto sonuc = v | std::views::filter([](int x){ return x % 2 == 0; })
               | std::views::take(3);
for (int x : sonuc) std::cout << x << " ";
```

Çıktı:

Bölüm F: Açık Uçlu Sorular

1. `std::optional` ile hata kodu döndürme (ör. `return -1;`) yaklaşımını karşılaştırın. Hangi durumlarda hangisi tercih edilir? **6 puan**

2. Aşağıdaki OBYS senaryosunu gerçekleştiren bir C++ programı yazın: **8 puan**

   - `std::map<int, std::string>` ile öğrenci deposu oluşturun (numara → isim)
   - `std::ranges::sort` ve projeksiyon ile isimlere göre sıralı liste elde edin
   - `std::optional<std::string>` döndüren bir arama fonksiyonu yazın
   - Bulunamayan öğrenci için `value_or("Bilinmiyor")` kullanın

3. Aşağıdaki kodda bir hata vardır. Hatayı bulun ve düzeltilmiş hâlini yazın. **5 puan**

```cpp
std::string veriAl() {
    return "gecici veri";
}

std::string_view sv = veriAl(); // ???
std::cout << sv << '\n';
```

Başarılar!